

Prueba Técnica. Perfil Practicante

Contexto

Esta prueba evalúa conocimientos fundamentados y pensamiento lógico aplicados a un entorno real de fábrica de software. Aunque el nivel es junior, se espera una aproximación estructurada, claridad en código y uso correcto de buenas prácticas.

La prueba está diseñada para resolverse en **3-4 horas**, y se recomienda entregar las respuestas a través de:

- Un repositorio Git (GitHub)
- Con un README donde el estudiante explique sus soluciones

Instrucciones

1. Crea un repositorio y comparte acceso.
2. Cada ejercicio debe estar ubicado en carpetas separadas dentro del repo (ejercicio-01, ejercicio-02, etc.).
3. El código debe ser claro, comentado cuando sea necesario.
4. Puedes usar HTML, CSS, JS nativo, y si lo deseas agregar Vue.js o Java en los ejercicios opcionales.
5. Utiliza commits pequeños, con mensajes claros y descriptivos.
6. Tiempo recomendado: **3 a 4 horas**.

1. HTML + CSS. Construcción de una mini landing page responsive (10%)

Crea una página con:

- Un encabezado con menú (no funcional).
- Una sección central con un título, descripción y un botón.
- Una cuadrícula de **3 tarjetas** de servicios o funcionalidades.
- Debe adaptarse a pantallas móviles usando flex o grid.

Lógica evaluada

- Estructuración semántica.
- Pensamiento en layouts.
- Responsividad sin frameworks.

Criterios

- Uso de flex o grid.
- Buen manejo de breakpoints.
- Código limpio y organizado.

2. JS. Manipulación del DOM con interacción lógica real (10%)

Debes crear una lista dinámica de tareas:

- Un input para escribir una tarea.
- Un botón para agregarla a la lista.
- Cada tarea debe tener:
 - Botón de eliminar
 - Botón de marcar como completada (cambia estilo)

Lógica evaluada

- Renderizado dinámico.
- Control de estado.
- Delegación de eventos.

Criterios

- Sin usar frameworks.
- Uso correcto de eventos y DOM.

3. Lógica. Problema de transformación de datos (10%)

Dado el siguiente arreglo:

```
const estudiantes = [  
  { nombre: "Ana", notas: [3.2, 4.1, 3.9] },  
  { nombre: "Luis", notas: [2.8, 3.0, 3.5] },  
  { nombre: "Marta", notas: [4.5, 4.6, 4.9] }  
];
```

Transforma los datos en un nuevo arreglo como este:

```
[  
  { nombre: "Ana", promedio: 3.73 },  
  { nombre: "Luis", promedio: 3.1 },  
  { nombre: "Marta", promedio: 4.67 }  
]
```

El resultado debe ser impreso en consola.

Lógica evaluada

- Reducción de arrays.
- Promedios.
- Transformaciones funcionales.

4. JS. Lógica condicional avanzada (10%)

Escribe una función que reciba una hora en formato 24h ("14:35") y convierta a formato 12h ("2:35 PM").

Lógica evaluada

- Parsing
- Manipulación de strings
- Condiciones complejas

5. Lógica. Algoritmo de filtrado múltiple (10%)

Dado este arreglo:

```
const productos = [  
  { nombre: "Café", precio: 1500, categoria: "bebida" },  
  { nombre: "Arepá", precio: 2000, categoria: "comida" },  
  { nombre: "Té", precio: 1000, categoria: "bebida" },  
  { nombre: "Sandwich", precio: 5500, categoria: "comida" }  
];
```

Crea una función que permita filtrar por:

- un rango de precios
- una categoría opcional

Y retorne todos los productos que coincidan con los filtros.

Lógica evaluada

- Filtrado avanzado
- Combinar condiciones múltiples
- Uso de objetos como parámetros

6. Lógica. Mini API mock usando JSON y JS (10%)

Crea un archivo **JSON** con esta estructura:

```
[  
  { "id": 1, "nombre": "Producto A", "precio": 5000 },  
  { "id": 2, "nombre": "Producto B", "precio": 7500 }  
]
```

Luego crea un script JS que:

- Cargue este JSON (local).

- Busque un producto por ID.
- Retorne un mensaje tipo:
"El Producto B cuesta 7500"

Lógica evaluada

- Lectura de archivos (fetch local).
- Búsqueda eficiente.

7. Problema: Sistema simple de gestión de tareas

Implementa un pequeño programa que permita:

1. Crear tareas, cada una con la siguiente estructura:
 - ID auto-generado
 - Título
 - Descripción
 - Estado (Pendiente, En progreso, Completada)
2. Listar tareas (todas o filtradas por estado).
3. Actualizar el estado de una tarea usando su ID.
4. Eliminar una tarea.

Requisitos técnicos obligatorios

- Debe usarse una **estructura de datos** para almacenar las tareas (array, lista, mapa, etc.).
- El código debe demostrar:
 - uso de funciones o métodos,
 - claridad en nombres,
 - separación entre lógica y datos,
 - estructura limpia y comentada.

8. Mini reto Colaborativo / Simulación de Trabajo en Equipo (10%)

Simula que estás trabajando con un equipo para el ejercicio anterior (Ejercicio # 7). Debes escribir un archivo en el repositorio llamado:

COLABORATION_SIMULATION.md

En este archivo describe:

1. **Cómo coordinarías** este desarrollo si trabajas con otra persona (máx. 8 líneas).
2. **Cómo dividirías las tareas** del proyecto.
3. **Cómo manejarías un desacuerdo técnico simple.**
4. **Cómo dejarías evidencia en commits claros** (dar 2 ejemplos de mensajes de commit).

Este archivo evalúa tu capacidad de **comunicación, organización y pensamiento colaborativo** sin involucrar a otra persona real.

9. Vue.js (Opcional pero recomendado). Mini componente con estados (10%)

Construye un componente que:

- Reciba una lista de usuarios por **props**.
- Muestre la lista.
- Tenga un input que filtre usuarios por nombre en tiempo real.

Lógica evaluada

- Props
- Computed properties
- Bindings reactivos

10. Java (Opcional). Clase + método de negocio (10%)

Crea una clase llamada Student con:

- nombre

- id
- lista de notas

Y un método getPromedio().

Luego crea una clase Main que:

- instancie al menos 2 estudiantes
- imprima sus promedios

Lógica evaluada

- POO
- Encapsulamiento
- Métodos

Distribución de puntaje

Ejercicio	Puntaje
1	10%
2	10%
3	10%
4	10%
5	10%
6	10%
7	10%
8	10%
9 (Vue.js opcional)	10%
10 (Java opcional)	10%